

3)

Code

```
GNU nano 8.7.1 Practical-03.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n===== CHILD PROCESS =====\n");
        printf("Child PID      : %d\n", getpid());
        printf("Parent PID       : %d\n", getppid());

        printf("Child State      : Running\n");

        sleep(10);

        printf("Child State      : Terminated\n");
        exit(0);
    }
    else
    {
        printf("\n===== PARENT PROCESS =====\n");
        printf("Parent PID       : %d\n", getpid());
        printf("Child PID        : %d\n", pid);

        printf("Parent State      : Waiting\n");

        wait(NULL);

        printf("Child Finished.\n");
        printf("Parent State      : Running\n");
        printf("Parent Terminated\n");
    }

    return 0;
}
```

Output:

===== PARENT PROCESS =====

Parent PID : 669

Child PID : 670

Parent State : Waiting

===== CHILD PROCESS =====

Child PID : 670

Parent PID : 669

Child State : Running

Child State : Terminated

Child Finished.

Parent State : Running

Parent Terminated